

Introduction to Scalable Systems Assignment - I

Guruprasad Sajjan

September 20, 2025

1 Problem 1

1.1 Problem Statement

- We are given a list of parcels, where each parcel is represented by an `id` and a `weight`.
- Two parcels are considered duplicates if they have the same `id` but different `weight`.

Goal: Detect all duplicate parcels and return the minimum weight occurrence of each duplicate parcel in sorted order of ids.

1.2 Solution Approach

1. Use a hash map (`unordered_map`) to store information for each parcel id.
 - Key = `parcel_id`
 - Value = (min_weight, max_weight)
2. Traverse all parcels:
 - If the id is new, initialize (w, w) .
 - If the id exists, update both the minimum and maximum weight.
3. After processing, a parcel id is considered duplicate only if
$$min_weight \neq max_weight$$
4. For each such id, add $[id, min_weight]$ to the result.
5. Sort the result by id before returning.

1.2.1 Reasoning

Here Neive approach can be to store all distinct weights for each id and then check if there are more than one unique weights. But this will waste lot of space.

To say parcel is duplicate we just need to know if there are at least two unique weights for that parcel id.

For that we can just store minimum and maximum weight for each id. This will not just help to say parcel id is duplicate or not but also help to get the minimum weight for that id and range of weights for that id.

Another way can be to store minimum weight and a boolean variable to say if we have seen more than one unique weight or not. But this will not help to get range of weights for that id. But reduces space (Since Boolean type is 1 Byte, But Int is 4 Byte).

1.3 Time and Space Complexity Analysis

- Time Complexity:
 - Traversing all parcels: $O(n)$, where n is the number of parcels.
 - Sorting the result: $O(k \log k)$, where k is the number of duplicate ids.
 - Overall: $O(n + k \log k)$ since $k \ll n$, we can say $O(n)$
- Space Complexity:
 - Hash map storage: $O(m)$, where m is the number of unique parcel ids.
 - Result storage: $O(k)$
 - Overall: $O(m + k)$
- Best Case: If all parcels have the same id, there are no duplicates, and sorting is trivial. Total time is $O(n)$. and space is $O(1)$.
- Worst Case: If a large number of parcel ids are duplicates (e.g., $k = n/2$), the result list becomes sizable. Sorting the result then takes $O(k \log k) = O(n \log n)$ time, making the overall time complexity $O(n \log n)$. Space complexity in this case is $O(n)$.

1.4 Experimental Setup

We are given a list of parcels, where each parcel is represented by its id and weight. number of duplicates can depend on range of weights and range of ids that we sample from. So we will fix range of weights and ids and vary number of parcels to see how it affects time and space complexity.

- Range of ids and weights: $[10^2, 10^3, 10^4, 10^5, 10^6, 10^7]$
- Number of parcels: $[10^2, 10^3, 10^4, 10^5, 10^6, 10^7]$

1.5 Empirical Observations

- Time complexity grows linearly with the number of parcels and is not affected by id or weight range.
- Space complexity increases with the range of ids, as more unique ids require more storage.

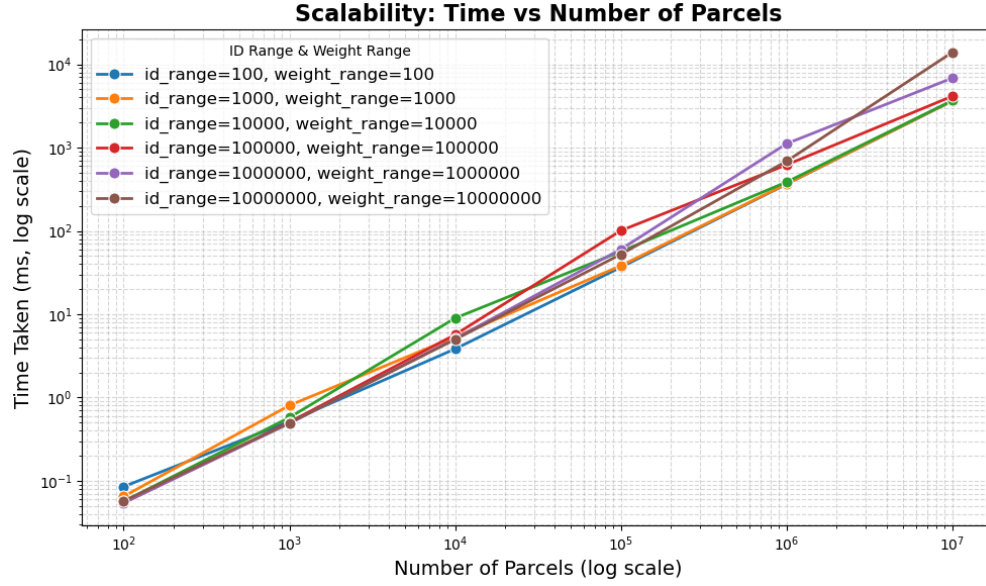


Figure 1: Time Complexity vs Number of Parcels

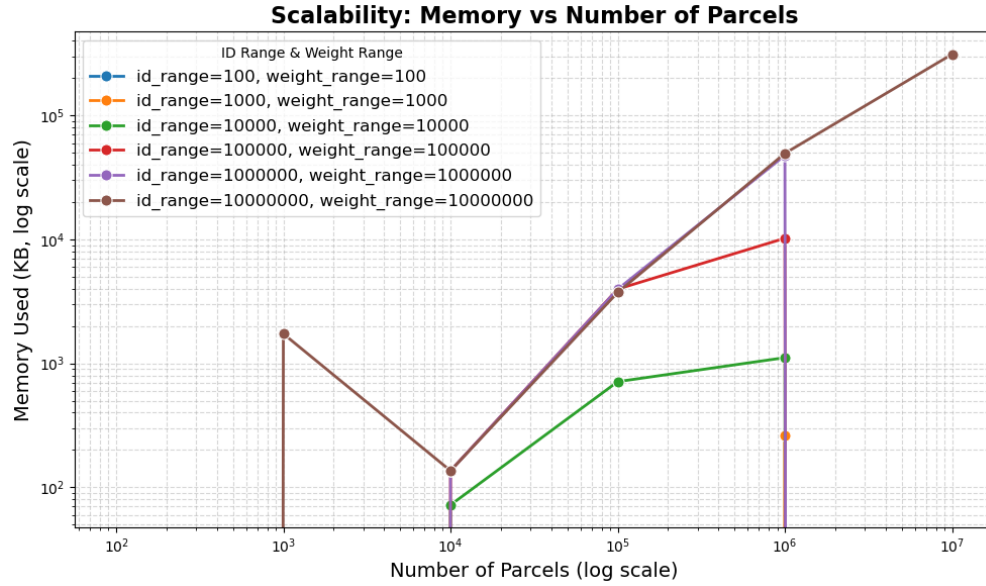


Figure 2: Space Complexity vs Number of Parcels

2 Problem 2

2.1 Problem Statement

- We have a binary conveyor network represented as a binary tree.
- Each junction is a node that splits into two belts (left and right).
- The topmost junction (root) is numbered 0, and junctions are numbered in level-order traversal.
- Parcels are routed to loading junctions (leaf nodes).
- We have a mapping of parcels to loading junctions and a list of damaged parcel IDs (query list).
- The goal is to find the highest-numbered junction ID where all parcels in the query list were present.

2.2 Solution Approach

1. Rebuild the binary tree from the given preorder and inorder traversals.
2. Collect leaf node ids during tree construction and Sort them by there node id.
3. Create a mapping from each parcel id to the leaf node ids where it is present.
4. Use backtracking to find and store the path from the root to each leaf node in a map (key = leaf id, value = path from root to leaf).
5. For each query:
 - If all parcels in the query present at same leaf node , then that leaf node is the answer.
 - For each query, perform binary search on the paths of each parcel in the query to find the highest numbered junction that contains that parcel.
(How , What is search space ? and what is monotonic condition here ?)
 - **Search space:** Path to leaf node with minimum length among all parcels in the query.
 - **Monotonic condition:** If all parcels in the query are present at junction id mid, then they will also be present at all junctions from 0 to mid.
 - If the condition is satisfied, move to the right half of the search space to find a higher junction. Otherwise, move to the left half.
 - The result for the query is the highest junction id that is common to all parcels in the query.
6. Return the results for all queries.

Note: The paths are always in increasing order due to level-order numbering of nodes.

2.2.1 Reasoning

Here main aim is to find the highest numbered junction that contains all parcels in the query. Equivalently,

Highest numbered common node in path(s) to all parcels in the query.

This point of view helps to see the search space as True for all nodes from 0 to that node and False for all nodes after that node. So we can use binary search to find that node. The 0^{th} node will always be common to all parcels in the query.

2.3 Time and Space Complexity Analysis

Let N = Number of nodes in tree, h = Height of the tree, n = Number of leaf nodes
 k = Number of queries, q = Number of parcels in each query, p = Number of unique parcels
 m = Number of parcels at each leaf node

- Time Complexity:
 - Rebuilding the tree: $O(N)$
 - Collecting leaf ids and creating parcel to leaf mapping: $O(n \cdot m)$
 - Backtracking to find paths: $O(N)$
 - For each query, performing binary search on paths: $O(k \cdot q \cdot \log h)$
 - Overall: $O(N + n \cdot m + N + k \cdot q \cdot \log h)$
- Space Complexity:
 - Storing the tree: $O(N)$
 - Parcel to leaf mapping: $O(p)$ (Assuming at least one parcel at each leaf)
 - Paths storage: $O(n \cdot h)$
 - Overall: $O(N + p + n \cdot h)$
- Best Case: If we have left skewed or right skewed tree, height h becomes $O(N)$. All parcels in a query must be present at the same leaf node, we can directly return that leaf node as the answer.
This results in a time complexity of $O(N + m + N + k \cdot q)$ and space complexity of $O(N + p + N)$.
- Worst Case: Complete binary tree, height h is $O(\log N)$. Number of leafs n is $O(N)$.
Space complexity becomes $O(N + p + N \log N)$.
Time complexity becomes $O(N + m + N + k \cdot q \cdot \log(\log N))$.

We can say for constant number of nodes in tree, If increase height of tree, then number of leaf nodes decreases and vice versa.

2.4 Experimental Setup

Constraints are given in terms of number of leaf nodes, number of parcels at each leaf node and number of queries. constraints :

- Number of leaf nodes (n): $[10^2, 10^3, 10^4, 10^5, 10^6]$
- Number of parcels at each leaf node (m): uniformly distributed from 1 to 100
- Number of queries (k): $[10^2, 10^3, 10^4, 10^5]$
- Number of parcels in each query (q): uniformly distributed from 1 to 100

For each pair of (n, k) , we will generate random binary tree with n leaf nodes and random parcels at each leaf node. Then we will generate k random queries and run the algorithm to get average time and space.

2.5 Empirical Observations

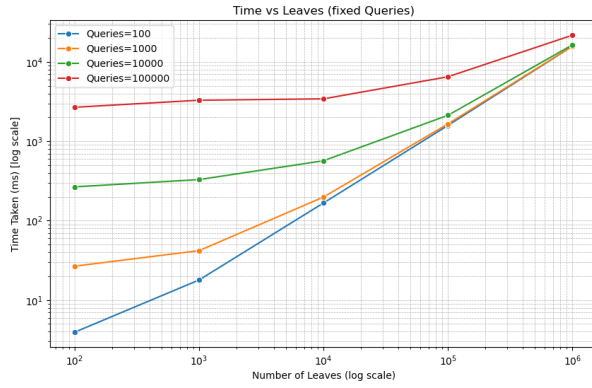


Figure 3: Time vs Number of Leaf Nodes

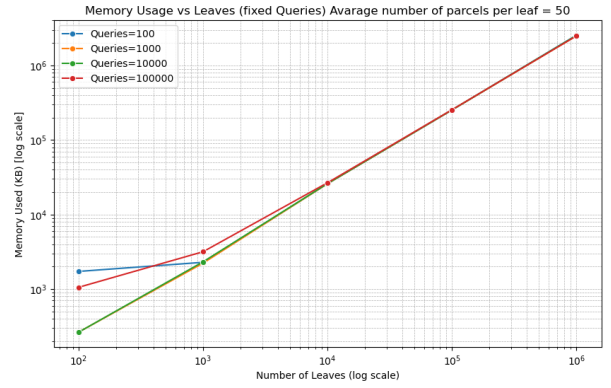


Figure 4: Space vs Number of Leaf Nodes

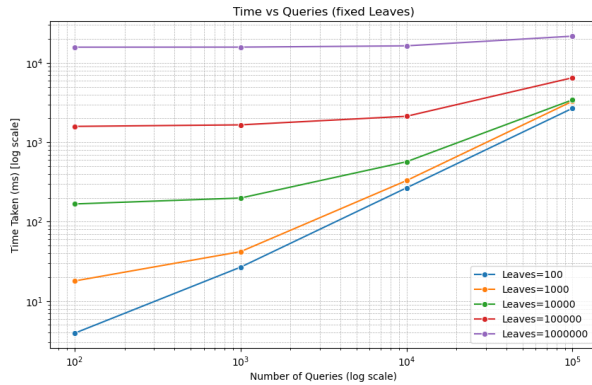


Figure 5: Time vs Number of Queries

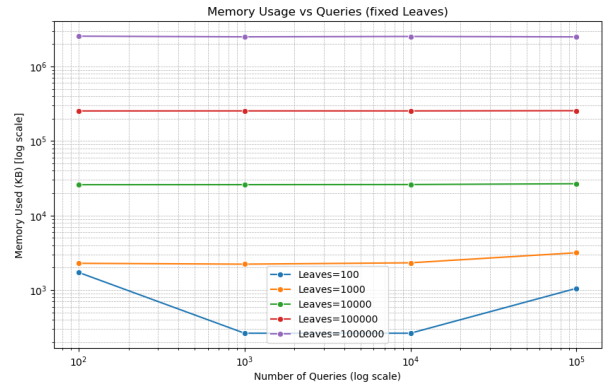


Figure 6: Space vs Number of Queries

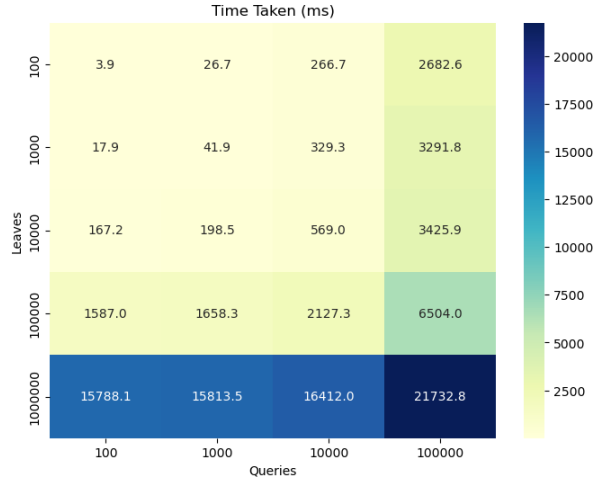


Figure 7: Time Complexity Heatmap

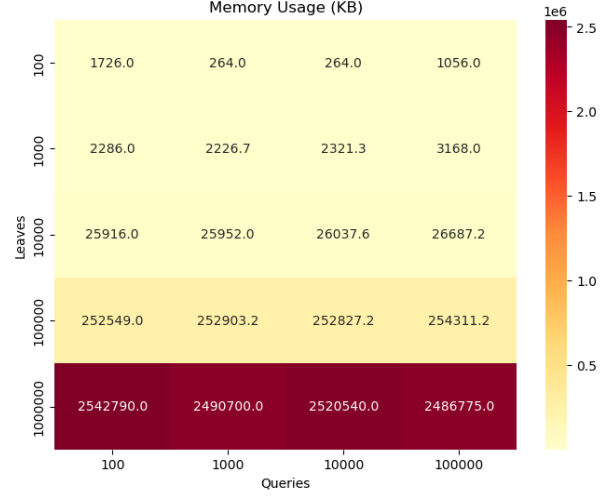


Figure 8: Space Complexity Heatmap

- Time complexity increases with the number of leaf nodes and number of queries, as expected from the theoretical analysis.
- Space complexity increases with the number of leaf nodes due to path storage, and also depends on the number of parcels.
- As we can clearly see Number of queries has no effect on space complexity. But time complexity increases linearly with number of queries.

2.5.1 Effect of skewed tree on time and space complexity

Leaves	Total Queries	Total Nodes	Parcels	Time (ms)	Memory (KB)
1	100,000	100	44	380.66	792
1	100,000	1,000	42	365.86	1,056
1	100,000	10,000	31	283.34	2,640

Table 1: Performance Analysis for Skewed Trees

- For highly skewed trees (single leaf), the time complexity remains relatively constant regardless of total nodes.
- Memory usage increases with total nodes due to path storage at leaf.
- This confirms our best-case analysis for skewed trees.

2.5.2 Effect of height on time and space complexity in complete binary tree

Height	Total Queries	Leaves	Total Parcels	Time (ms)	Memory (KB)
10	1,000	512	25,996	15.85	1,056
15	1,000	16,384	831,242	253.16	36,160
20	1,000	524,288	26,482,481	7,876.69	1,177,530
10	10,000	512	23,566	92.36	1,076
15	10,000	16,384	828,568	340.95	36,160
20	10,000	524,288	26,469,294	8,003.74	1,177,160
10	100,000	512	25,817	854.20	2,112
15	100,000	16,384	830,905	1,212.17	37,044
20	100,000	524,288	26,477,160	9,210.79	1,178,400

Table 2: Performance of Complete binary tree with varying heights and number of queries

- As the height of the complete binary tree increases, both time and space complexity increase significantly.
- This aligns with our theoretical analysis, where height h affects both complexities.
- The number of queries has a linear impact on time complexity but does not affect space complexity.
- This confirms our worst-case analysis for complete binary trees.
- For constant number of nodes in tree, If increase height of tree, then number of leaf nodes decreases and vice versa.

3 Problem 3

3.1 Problem Statement

- We have a weighted graph representing cities (nodes) and roads (edges with travel time).
- Two trucks start from different cities (node 1 and node n) and can refuel at metro cities (k nodes) to double their speed.
- **Assuming both trucks start simultaneously.**
- Refueling takes 0 time and can be done at most once by each truck.
- The goal is to find the earliest possible time for the two trucks to meet in **any city**.
- If the trucks cannot meet due to disconnection between nodes 1 and n, return -1.

3.2 Solution Approach

1. Use Disjoint Set Union (DSU) to check if nodes 1 and n are connected.
If not, return -1.
2. Build an adjacency list representation of the graph from the edges.
3. Convert the list of metro cities into a set for $O(1)$ lookup.
4. Implement a modified Dijkstra's algorithm that tracks two states for each node: whether the truck has refueled or not.
5. Run this modified Dijkstra's algorithm from both starting nodes (1 and n) to get the shortest travel times to all other nodes, considering both refueled and non-refueled states.
6. For each node in the graph, calculate the maximum time taken by both trucks to reach that node (considering both states) and keep track of the minimum of these maximum times across all nodes.
7. Return this minimum time as the earliest possible meeting time.

3.2.1 Reasoning

We can use DSU to check if nodes 1 and n are connected. Because given graph is undirected and cost of computation is lesser than main algo. If they are not connected, we can directly return -1.

But why Dijkstra's algorithm works here?

Because *Path followed (Time taken) by one truck does not affect the path followed by other truck*. So we can independently find the shortest time taken by each truck to reach all nodes.

We maintain two states for each node, whether the truck has refueled or not. This is because refueling changes the travel time for subsequent roads.

Since there no restriction on which node to meet, we can check for all nodes and find the minimum of maximum time taken by both trucks to reach that node. That will be our answer.

3.3 Time and Space Complexity Analysis

Let N = Number of nodes, M = Number of edges, K = Number of metro cities

- Time Complexity:
 - DSU operations: $O(M)$
 - Building the graph: $O(M)$
 - Modified Dijkstra's algorithm: $O((N + M) \log N)$ for each run, so $O(2(N + M) \log N)$ overall.
 - Checking all nodes for meeting point: $O(N)$
 - Overall: $O(M + M + 2(N + M) \log N + N) = O((N + M) \log N)$
- Space Complexity:
 - Graph storage: $O(N + M)$
 - DSU storage: $O(N)$ (We can free this after connectivity check)
 - Distance arrays for both runs of Dijkstra's: $O(2N)$
 - Metro cities set: $O(K)$
 - Overall: $O(N + M + K) = O(N + M)$ (since always $K \leq N$)
- Best Case: If nodes 1 and n are not connected, we can return -1 after DSU check. Time complexity is then $O(M)$ and space complexity is $O(N)$.
- Worst Case: If the graph is dense (i.e., $M \approx N^2$), the time complexity becomes $O((N + N^2) \log N) = O(N^2 \log N)$. Space complexity remains $O(N + M + K)$.
- The number of metro cities K affects space complexity but has a minimal impact on time complexity since lookups in the metro cities set are $O(1)$.
- The performance of Dijkstra's algorithm is influenced by the graph's density (i.e., the ratio of edges to nodes). In sparse graphs, it performs closer to $O(N \log N)$, while in dense graphs, it approaches $O(N^2 \log N)$.

3.4 Experimental Setup

We will vary the number of nodes and edges to see how it affects time and space complexity.

- Number of nodes (N): $[10^2, 10^3, 10^4, 10^5, 2 \cdot 10^5]$
- Edge count based on graph density: $[0.1, 0.35, 0.5, 0.65, 0.7]$ of maximum possible edges.
- Metro cities: $[0.01, 0.1, 0.35]$ of total nodes, chosen randomly.
- For each (N , density), generate a random graph and metro cities, then compute average time and space complexity.

3.5 Empirical Observations

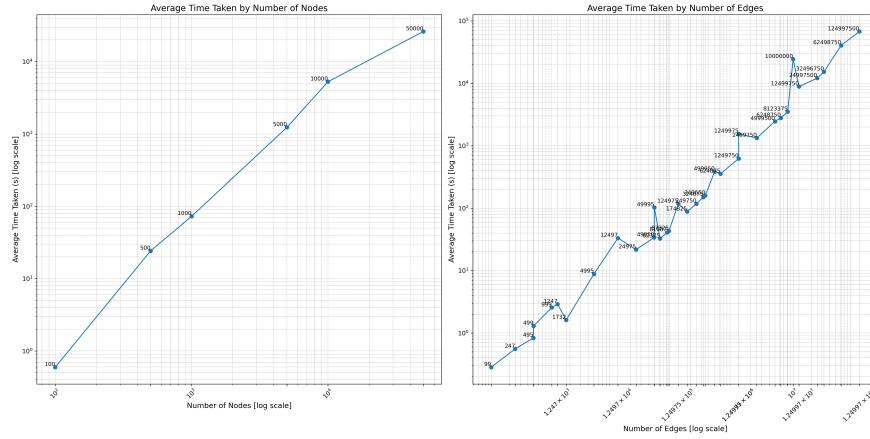


Figure 9: Time Complexity vs Number of Nodes for Different Densities

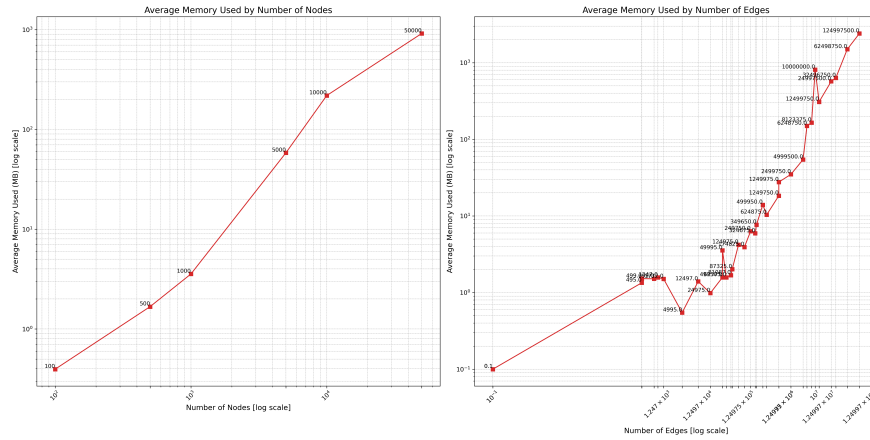


Figure 10: Space Complexity vs Number of Nodes for Different Densities

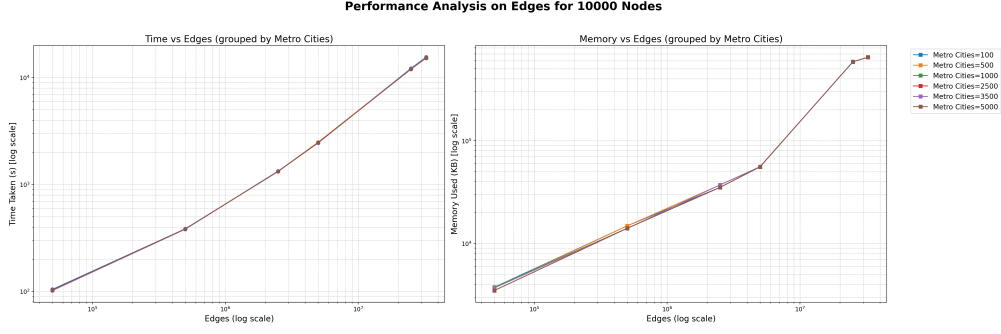


Figure 11: Time and Space Complexity vs Number of Edges for Fixed Nodes

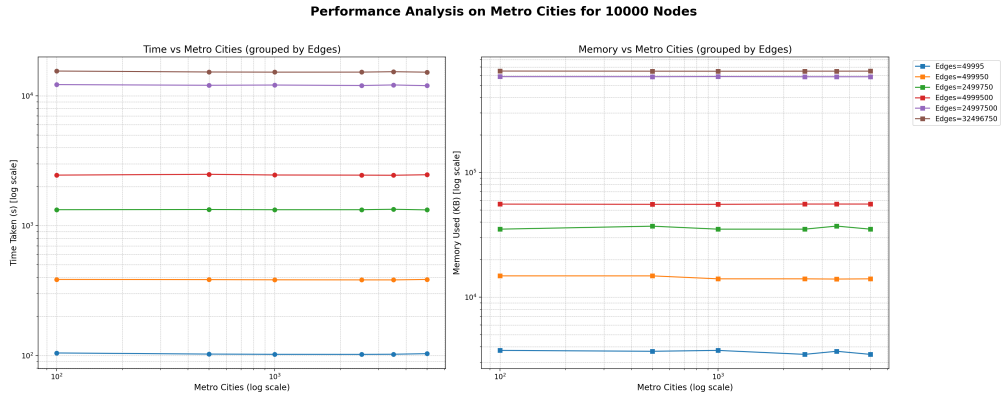


Figure 12: Time and Space Complexity vs Number of Metro Cities for Fixed Nodes

- Time complexity and space complexity increases with the number of nodes and edges, as expected from the theoretical analysis. from first two plots.
- For fixed number of nodes, We can see linear increase in time, But Space complexity increasing even more rapidly with number of edges as expected. from third plot.
- From fourth plot, we can clearly see that number of metro cities has no effect on time and space complexity.
- The performance of Dijkstra's algorithm is influenced by the graph's density. In sparse graphs, it performs closer to $O(N \log N)$, while in dense graphs, it approaches $O(N^2 \log N)$.